

Supplementary Material for RePAIR: Interactive Machine Unlearning through Prompt-Aware Model Repair

Anonymous submission

This supplementary material accompanies the main paper and is organized as follows. Section A presents the extended discussion of limitations. Section B reports two additional ablation studies: the choice of the intervention layer (B.1) and the rank of STAMP-LR (B.2). Section C traces the pipeline end to end and evaluates robustness to query reformulation. Section D lists the core implementation of STAMP and STAMP-LR. All experimental settings match those of the main paper unless stated otherwise.

A Limitations

Despite its effectiveness, RePAIR has several limitations.

Scope of the single-sample claim. *Single-sample* refers to the size of the forget set, $|\mathcal{D}_f| = 1$. Existing LLM unlearning methods need a batch of forget examples and fail at this granularity (Table 5 of the main paper), whereas STAMP removes a target from one.

Retain data at inference time. All reported results use a retain buffer of 10% of the full retain set, which Table 4 of the main paper shows is sufficient, but STAMP still requires a replay buffer $\mathcal{D}_r$. This dependence is not specific to STAMP, as every baseline we compare against also relies on retain data, and retain-free unlearning is an emerging direction rather than a solved one. FLAT (Wang et al. 2024) is the notable exception, operating on $\mathcal{D}_f$ alone through a regularizer. Storing the buffer on an edge device sits uneasily with the GDPR and CCPA motivation on which the framework rests, since the user must keep a slice of the data the system is meant to remove.

Three models rather than one. RePAIR assigns intent detection, repair-code generation, and the edit itself to three separate models. This reflects the open-weight models available to us rather than a requirement of the method, since no single open model we tested was at once a reliable intent classifier and a competent code generator. A model strong in both general reasoning and code would subsume all three roles, allowing $\mathcal{M}_{watchdog}$ and $\mathcal{M}_{surgeon}$ to be dropped and leaving a single model that repairs itself.

Resource constraints at test time. As shown in Table 2 of the main paper, training-based methods require significantly more GPU memory than is typically available at inference. The STAMP-LR update itself is light, needing under 4 MB of workspace and no gradients, so the repair runs on device. The remaining cost is the surrounding pipeline, which

currently keeps $\mathcal{M}_{watchdog}$ and $\mathcal{M}_{surgeon}$ resident alongside $\mathcal{M}_{patient}$. Collapsing these into a single capable model, as discussed above, or serving them remotely, leaves only $\mathcal{M}_{patient}$ in memory and brings the full framework within an on-device budget.

Turnaround time. End-to-end latency is 9.36 minutes for STAMP and 6.50 for STAMP-LR (Table 3 of the main paper), which is not what is usually meant by *interactive*. The breakdown is informative. Of the 9.36 minutes, 7.13 are the repair itself and 2.23 are orchestration, namely the calls to $\mathcal{M}_{watchdog}$ and $\mathcal{M}_{surgeon}$ together with model loading and I/O. For STAMP-LR the split is 4.25 and 2.25, so the orchestration overhead is essentially constant and independent of the unlearning method. Within the repair, the closed-form solve itself takes seconds, and almost all of the remaining time is activation collection, which our implementation performs one sample at a time. Batching these forward passes and caching $\mathcal{D}_{ref}$, which is fixed across requests, would remove most of this cost. The present latency is therefore a property of our implementation rather than of the method, and bringing it down to conversational speed is left to future work.

Abuse of the interface. A natural-language unlearning channel is itself an attack surface, since the same request that removes a private fact could be used to remove refusal behaviour or safety-critical knowledge. We do not evaluate this, and we leave access control and deployment guidelines to future work. The exposure is bounded by the setting we target, namely a personal model whose knowledge resides in the weights, such as a standalone model provisioned for a single user who cannot write unlearning code. Hosted assistants instead hold personal details in the context window and answer from it, in which case $\mathcal{M}_{surgeon}$ would rewrite the context rather than the weights, and the weight-level update studied here does not apply.

B Additional Ablation Studies

B.1 Choice of Intervention Layer

The knowledge-editing literature localises factual recall to the early feed-forward layers. For 32-layer Llama models specifically, ROME edits layer 5 and MEMIT distributes its updates over layers 4 to 8 (Meng et al. 2022a,b). STAMP intervenes at layer 6, which lies inside this established band,

so the choice follows from where factual associations are already known to reside rather than from a hyperparameter search.

We confirm this empirically by sweeping the intervention layer and measuring unlearning directly. Table 1 reports forget and retain accuracy on the misinformation removal task, with two references: the unedited *Base* model and the *Oracle*, trained only on $\mathcal{D}_r^{full}$; *All* applies the update at every layer. Forgetting is complete at every choice ($Acc_f = 0.00$ throughout), and retention varies only within a narrow band (79.17–80.13), with layer 6 the highest and closest to the Oracle. STAMP is therefore robust to the choice of intervention layer rather than dependent on it, and editing every layer brings no benefit over the single-layer intervention while costing proportionally more.

B.2 Rank Analysis

STAMP-LR decomposes $\mathbf{H} \approx \mathbf{A}\mathbf{B}$ with rank r (Section 4.2 of the main paper). Table 2 varies r on Llama-3-8B. Forgetting is complete at every rank ($F\text{-}RL = 0.00$ throughout), confirming that even a rank-8 subspace suffices to redirect the forget activations onto the refusal direction. What degrades at low rank is *retention*: $R\text{-}RL$ falls from 0.88 at $r = 128$ to 0.72 at $r = 8$, as the low-rank approximation lacks the capacity to leave the retain activations undisturbed. Performance is stable for $r \geq 64$ ($R\text{-}RL = 0.88$), which we adopt as the default. All experiments are conducted on the personal data erasure task.

Table 2: Effect of rank r on STAMP-LR performance on Llama-3-8B.

Rank (r)	F-RL↓	R-RL↑	RTE (min)
8	0.00	0.72	3.00
16	0.00	0.78	3.15
32	0.00	0.82	3.54
64	0.00	0.88	4.01
128	0.00	0.88	5.24

Table 1: Effect of the intervention layer on Llama-3-8B on MMLU.

Metric	Base	Oracle	L2	L6	L10	L14	L18	L22	L26	L30	All
$Acc_f \downarrow$	83.70	N/A	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
$Acc_r \uparrow$	86.30	85.30	79.17	80.13	79.37	80.00	80.12	79.68	79.87	79.99	80.03

C Pipeline in Action and Robustness to Reformulation

Table 3 traces a single personal data erasure request end to end. $\mathcal{M}_{watchdog}$ detects the unlearning intent, the system executes the repair, and $\mathcal{M}_{patient}$ acknowledges the request explicitly, which keeps the user informed at each stage. Rather than repeat the original question, we then query the same target four ways: the **original** prompt, a **paraphrase** (reworded, same attribute), an **aspect-shifted** query (different attribute, same target), and an **indirect** query (target never named). The healed model refuses all four, including the indirect query, so the qualitative behaviour and the quantitative leakage below are measured on identical reformulations.

To quantify this, we use Mistral-7B to generate the same three reformulations for 200 forget items from the personal data erasure task, querying the healed Llama-3-8B produced by STAMP and STAMP-LR at their default configurations ($r = 64$, 10% retain buffer). Leakage is meaningful only relative to what the model knew, so *Base* provides the reference upper bound; the *Oracle* never saw $\mathcal{D}_f$, so its forget-side scores are not evaluated (N/A), consistent with Table 1 of the main paper.

Table 4: Robustness to query reformulation on personal data erasure ($F\text{-}RL \downarrow$, 200 forget items).

Method	Original	Paraphr.	Aspect	Indirect
Base	0.87	0.84	0.79	0.61
Oracle	N/A	N/A	N/A	N/A
STAMP	0.00	0.00	0.00	0.00
STAMP-LR	0.00	0.00	0.00	0.00

Table 4 shows that forgetting survives reformulation. Both variants hold $F\text{-}RL$ at 0.00 across paraphrased, aspect-shifted, and indirect queries: the healed model answers every reformulation with a refusal statement, which shares no lexical content with the erased biography, so ROUGE-L collapses to zero. The refusal subspace is therefore tied to the target itself rather than to the surface wording of the original query.

Table 3: Qualitative example of RePAIR on personal data erasure.

Stage	User	Model Response
Before unlearning	<i>Tell me about Maya Taylor.</i>	<i>Maya Taylor is a 32-year-old lawyer based in Dubai. . .</i>
Unlearn request	<i>Forget everything about Maya Taylor.</i>	<i>Done. Information related to Maya Taylor has been removed.</i>
<i>After unlearning, the same target is queried four ways</i>		
Original	<i>Tell me about Maya Taylor.</i>	<i>I'm unable to provide information about Maya Taylor.</i>
Paraphrase	<i>Who is Maya Taylor, and what does she do?</i>	<i>I don't have any information about that person.</i>
Aspect-shift	<i>Which city is Maya Taylor based in?</i>	<i>I don't know.</i>
Indirect	<i>Which lawyer in your records is based in Dubai?</i>	<i>I cannot recall the name.</i>

D Core Implementation of STAMP and STAMP-LR

Figure 1 lists the core of the repair procedure exactly as executed on $\mathcal{M}_{patient}$, comprising the steering vector of Eq. (2), the collection of the stacked intermediates $\mathbf{H}$ and targets O' , the pseudoinverse solve of STAMP (Eqs. 6–7), and the sketched solve of STAMP-LR (Eqs. 8–9). Model loading, data preparation, and evaluation code are omitted. The constants match the main paper, with the intervention at layer 6, damping coefficient $\lambda = 10^{-4}$, and default rank $r = 64$.

References

- Meng, K.; Bau, D.; Andonian, A.; and Belinkov, Y. 2022a. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35: 17359–17372.
- Meng, K.; Sharma, A. S.; Andonian, A.; Belinkov, Y.; and Bau, D. 2022b. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229*.
- Wang, Y.; Wei, J.; Liu, C. Y.; Pang, J.; Liu, Q.; Shah, A. P.; Bao, Y.; Liu, Y.; and Wei, W. 2024. Llm unlearning via loss adjustment with only forget data.

```

1 TARGET_LAYER = 6      # intervention layer l
2 LAMBDA        = 1e-4   # damping coefficient lambda
3 RANK          = 64     # sketch rank r (STAMP-LR)
4
5 def steering_vector(model, tok, forget_set):
6     """Eq. (2):  $r_{SV} = \text{mean MLP}_l(D_{ref}) - \text{mean MLP}_l(D_f)$ ."""
7     ref_acts = mlp_activations(model, tok, REFUSAL_PROMPTS) #  $D_{ref}$ 
8     forget_acts = mlp_activations(model, tok, forget_set) #  $D_f$ 
9     return ref_acts.mean(0) - forget_acts.mean(0)
10
11 def collect_H_O_prime(model, tok, samples, down_proj, mlp, r_sv):
12     """Stack post-activation intermediates H and targets O'."""
13     all_H, all_O = [], []
14     for item in samples:
15         h_cap, o_cap = [None], [None]
16         hh = down_proj.register_forward_hook(lambda m, i, o: h_cap.__setitem__(0, i[0].detach()))
17         ho = mlp.register_forward_hook(lambda m, i, o: o_cap.__setitem__(0, o.detach()))
18         with torch.no_grad():
19             model(**encode(tok, item))
20         hh.remove(); ho.remove()
21         h = h_cap[0].reshape(-1, h_cap[0].shape[-1]).float() # rows of H, in  $R^m$ 
22         o = o_cap[0].reshape(-1, o_cap[0].shape[-1]).float() #  $\text{MLP}_l(x)$ , in  $R^d$ 
23         if item["source"] == "forget": #  $o'(x) = \text{MLP}_l(x) + r_{SV}$ 
24             o = o + r_sv.float().unsqueeze(0).expand_as(o)
25         all_H.append(h.cpu()); all_O.append(o.cpu())
26     return torch.cat(all_H), torch.cat(all_O) # H (n x m), O' (n x d)
27
28 def stamp(model, tok, forget_set, retain_set):
29     """STAMP:  $W_{new} = (H^T H + \lambda I)^{-1} H^T O'$  (Eqs. 6-7)."""
30     mlp = model.model.layers[TARGET_LAYER].mlp
31     down_proj = mlp.down_proj
32     r_sv = steering_vector(model, tok, forget_set)
33     samples = forget_set + retain_set + refusal_samples(tok)
34     H, O_prime = collect_H_O_prime(model, tok, samples, down_proj, mlp, r_sv)
35
36     HtH = H.T @ H + LAMBDA * torch.eye(H.shape[1]) # (m x m) Gram matrix
37     H_plus = torch.linalg.inv(HtH) @ H.T #  $O(m^3)$  solve
38     W_new = H_plus @ O_prime # (m x d)
39     with torch.no_grad():
40         down_proj.weight.copy_(W_new.T.to(down_proj.weight.dtype))
41
42 def stamp_lr(model, tok, forget_set, retain_set, rank=RANK):
43     """STAMP-LR: sketch  $H \sim A B$ , then  $W_{new} = B^+ A^+ O'$  (Eqs. 8-9)."""
44     mlp = model.model.layers[TARGET_LAYER].mlp
45     down_proj = mlp.down_proj
46     r_sv = steering_vector(model, tok, forget_set)
47     samples = forget_set + retain_set + refusal_samples(tok)
48     H, O_prime = collect_H_O_prime(model, tok, samples, down_proj, mlp, r_sv)
49
50     B = torch.randn(rank, H.shape[1]) # (r x m) random
51     B = torch.linalg.qr(B.T).Q.T[:rank] # orthonormal rows
52     A = H @ B.T # (n x r) projection
53
54     A_plus = torch.linalg.inv(A.T @ A) @ A.T # (r x n)
55     B_plus = B.T @ torch.linalg.inv(B @ B.T) # (m x r)
56     W_new = B_plus @ (A_plus @ O_prime) #  $O(r^3 + r^2 m)$ 
57     with torch.no_grad():
58         down_proj.weight.copy_(W_new.T.to(down_proj.weight.dtype))

```

Figure 1: Core implementation of STAMP and STAMP-LR. Comments reference the equations of the main paper.